Parametric Multimodal User Memory: Storing What Captions Cannot Carry

Bojie Li
Pine AI

Noah Shi
University of Washington

Abstract

A personalized AI agent needs a *user memory*: a persistent model of who its user is, accumulated across many encounters and consulted on each new one. Today that memory is almost always stored as *text*. The agent keeps transcripts of what was said and captions of what was seen, then retrieves them by similarity search over those words. This works for the *captionable* slice of a person ("my cat is named Bibi"), but it silently drops a second slice that no caption can hold. That second slice is the *perceptual* one: how a user's voice sounds, how their face reads across lighting and age, how tired they sound today against their own baseline. The moment such content is written down, the signal that tells one person from another is gone. We argue that perceptual user memory needs a different primitive, one that keeps the content in its native modality instead of projecting it through text.

We introduce **parametric multimodal user memory**. The agent stores each perception directly, as a row in a small per-modality bank attached to a frozen language model, and recalls it by attention at generation time, with no captioning step in between. Registering a new identity is a single tensor append with no training. Recall takes constant time no matter how many are stored. On **PerceptMem**, a new benchmark spanning faces, voices, painting style, room acoustics, and tone of voice, the mechanism edges raw embedding retrieval, though a similarity learned on the same encoder matches that recall gain, so its value is not a sharper neighbour but that the perception lives *inside* the model, with $\mathcal{O}(1)$ registration and composition with text. When look-alike distractors confuse the encoder, adversarially-aware training recovers most of the lost accuracy ($+14$ to $+71$ points). Crucially, this memory sits *alongside* text memory rather than replacing it, and ordinary text recall is preserved exactly. An agent with both remembers what its user said *and* what they were like.

Code: https://github.com/19PINE-AI/multimodal-user-memory

1 Introduction

When an AI agent "remembers" something about the person it serves, what does it actually keep? In almost every system today, the answer is *text*. Mem0 [Chhikara et al., 2025], MemoryLLM [Wang et al., 2024], MemGPT / Letta [Packer et al., 2023, Team, 2024], M3-Agent [Long et al., 2025], and the wider conversational-memory line (measured by benchmarks such as LongMemEval [Wu et al., 2024]) turn what the agent saw or heard into transcripts (via ASR [Radford et al., 2022]) and captions (via BLIP-2 [Li et al., 2023] or LLaVA [Liu et al., 2023]), drop those fragments into a sentence-encoder vector store [Reimers and Gurevych, 2019], and

search over them later. Personalized vision-language systems (MyVLM [Alaluf et al., 2024], Yo'LLaVA [Nguyen et al., 2024], MC-LLaVA [An et al., 2024], Online-PVLM [Bai et al., 2025], RAP [Hao et al., 2025]) keep the same textual spine: a *named* concept ("Bibi") is the handle, and the perception is an afterthought.

This is exactly right for one half of a person. "My cat is named Bibi." "My favourite restaurant is in Paris." "I am vegetarian." These are *captionable*: they pass through text intact and come back whole. But the captionable half is not the whole person. Consider what an attentive companion actually carries about someone:

- How their voice sounds when they say their own name, and how that differs from a stranger saying it.
- How their face reads today: a little older than last year's photo, but unmistakably them.
- How their brushwork looks: "this is by the painter from Tuesday, even though it's from a different period."
- What their kitchen sounds like: "you're calling from the same room as yesterday."
- How tired they sound today, relative to their baseline last week.

None of this is captionable in any useful sense. "A brown-haired man" does not tell two brown-haired men apart. "The user sounded tired" cannot separate today's tired from last week's tired. The instant we write these down, we throw away the very signal that made them discriminative. This is not a gap that a better caption closes. It is a property of the channel. **Text is a lossy medium for perceptual content, and the loss is fundamental to the modality.**

So a personalized agent's memory has two halves that want different homes. *Captionable* content (facts, preferences, propositions) belongs in a text store, where sentence-encoder similarity works beautifully. *Perceptual* content is different: how a face, a voice, a room, or a painting actually *is* needs a primitive that keeps it in its native modality. Today the second half has no such home. It is either flattened into a caption (and lost) or handed to a recognition tool that returns a bare label like "speaker 47", so the perception never reaches the model. The missing piece is a memory that stores a perception *as a perception*.

Our proposal. We introduce **parametric multimodal user memory**: instead of captioning a perception, the agent stores it directly, as a row in a small per-modality bank bolted onto a frozen language model. Each row pairs the raw encoder embedding of the perception (the key) with the model's own internal vector for a marker token (the value). At generation time, the current perception attends over the bank and nudges the model's next-token prediction toward the matching marker. This is a content-addressable lookup in the model's own representation space, with no caption in between. Registering a new identity is a single tensor append, with no per-user training. Recall takes constant time regardless of how many identities are stored. The idea borrows the shape of kNN-LM [Khandelwal et al., 2020] and Memorizing Transformers [Wu et al., 2022] (attention over a non-parametric bank), but aims it at a new target: the persistent perceptual identity of a user, registered in $\mathcal{O}(1)$. We call the resulting mechanism AttMem (for *attention memory*). Figures and tables refer to it by that name.

What this changes. Take a worked example. Ten voice clips from past sessions sit in the user's memory. A new clip arrives, and the agent must say which of the ten it is. The text

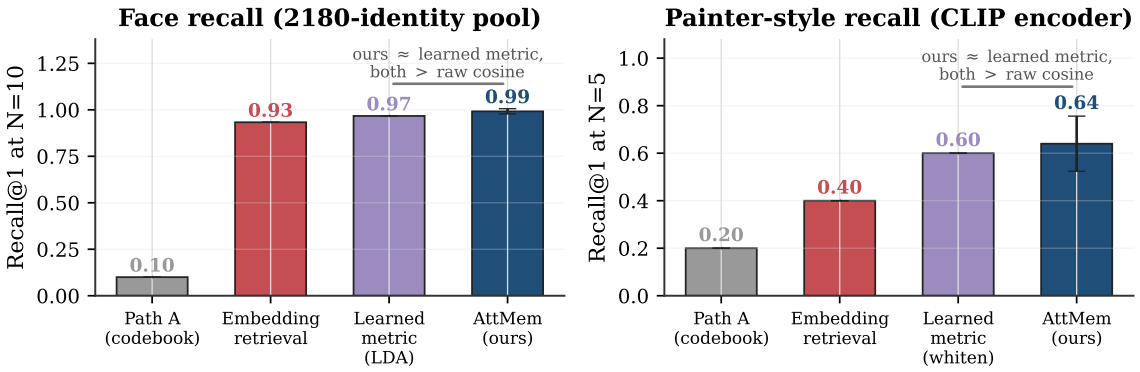


Figure 1. On two cross-condition perceptual-recall tasks, parametric multimodal memory edges raw embedding retrieval: a 2180-identity face pool (0.99 vs 0.93, p=0.006, 4 seeds) and painter-style recall (0.64 vs 0.40, 5 seeds). But a similarity *learned* on the same encoder (LDA for faces, within-class whitening for style) matches that lift, so the recall gain is metric learning rather than a uniquely sharper ruler (Appendix A). The parametric memory's distinct value is being in-model: the recalled identity is a token the language model conditions on, registered in constant time, composing with text recall. PATH A is a discrete-codebook predecessor, tuned to its ceiling before we abandoned it for continuous attention (Section 5).

route captions each clip ("a male voice, mid-30s, slight accent") and searches. The caption fits thousands of speakers, so the search is a coin flip. The parametric route stores each clip's voiceprint natively and matches on it, and it answers correctly. The same pattern holds for faces across years, paintings across periods, rooms across recordings, and tone against a personal baseline. And where we can pit it against the strongest text-free baseline directly (Figure 1), our memory edges raw embedding retrieval. A similarity learned on the same encoder matches that edge, so the recall gain is metric learning, not a property unique to our mechanism. What is unique is that the perception lives *inside* the model. It is recalled as a token the model can condition its generation on, not a label looked up in an external index.

Contributions.

- **A framing.** We argue that text-shaped memory is structurally incomplete for personalized agents, because it cannot carry perceptual user content, and that a parametric multimodal memory is the right primitive for the missing half (Sections 1–2).
- **A mechanism.** A bolt-on, content-addressable memory: per-modality banks of (encoder embedding, model value embedding) rows, read by attention at the model's output head. About 8M trainable parameters over a 3.1B frozen model, $\mathcal{O}(1)$ insertion, and constant $\sim$15 ms recall. That is 52$\times$ faster than feeding the same memory in as context, which runs out of memory entirely past ten thousand identities (Section 3).
- **A benchmark. PerceptMem**, five cross-condition perceptual sub-modalities (face, painter style, speaker, acoustic scene, and tone of voice), each chosen because text provably cannot carry its discriminating signal (Section 4).
- **An honest map.** We show exactly where the mechanism edges raw embedding retrieval, where a learned metric on the encoder erases that edge, where it merely ties, and where training the memory actually *hurts* (when the encoder is already perfect, there is nothing

to add). The recall gain over raw cosine is real but it is metric learning, so the lasting contribution is the in-model behaviour a retrieval index cannot replicate, not the recall number (Sections 5–6).

2 Four ways to remember a voice, and the one we build on

Why does the text route fail, not for lack of effort but structurally? Take the voice example in its hardest honest form: ten samples on file, a new one arriving cross-recording across different rooms and days, and the agent must name which of the ten it is. An agent has four options (Table 1). Three discard the very signal that tells one voice from another. Only the fourth keeps it, and it is the one we build on.

Table 1. Four routes for remembering a perception, plus ours (citations in text). Only routes that keep the raw perception can discriminate across conditions. Only ours reads it back in the language model's own representation space, which lets it exceed the encoder's similarity rather than inherit it.

Route	What it stores	Keeps percept?	Cross-condition	Register
Caption-and-search	text description	no	chance	$\mathcal{O}(1)$
Recognize-and-label	a bare label	no	label only	$\mathcal{O}(1)$
Per-concept tuning	tuned token/adapter	yes	strong	grad./id
Embedding retrieval	raw embedding	yes	encoder-limited	$\mathcal{O}(1)$
Parametric (ours)	embedding + LM value	yes	**beats encoder**	$\mathcal{O}(1)$

The three that discard the signal. *Caption-and-search* [Chhikara et al., 2025, Wang et al., 2024] writes the voice down ("a male voice, mid-30s, slight Eastern-European accent") and retrieves by similarity over the words. The description is true and useless. It fits thousands of speakers, so cosine over captions cannot separate ten users with even coin-flip reliability. *Recognize-and-label* [Long et al., 2025] calls a recognizer (ArcFace [Deng et al., 2019], ECAPA-TDNN [Desplanques et al., 2020]) and stores the label it returns ("speaker 47"). The model can recall that a label was assigned, but it never holds the perception and cannot compare today's voice against last week's. *Per-concept tuning* [Nguyen et al., 2024, Alaluf et al., 2024] fine-tunes a token or adapter per concept. It works, but each identity costs a second of gradient descent and yields a concept-specific artifact. That is the wrong economics for a companion registering dozens of characteristics per user, continuously.

The one that keeps it. *Embedding retrieval* skips the words, indexes the raw voiceprint, and retrieves by cosine over the encoder's vectors. It is the only text-free route that keeps the perception, and the strongest baseline we take seriously throughout. It succeeds exactly to the extent that the encoder is invariant across recording conditions, and fails to the extent that it is not. On a 2180-identity face pool it reaches 0.93 recall at ten identities. That is strong, but short of certainty, and it erodes as the pool grows.

Our mechanism takes this fourth route and does more with it. It stores the voiceprint as a key, but pairs it with the language model's *own* internal vector for a marker token as the value, and reads the pair back by attention inside the model. The recalled value is then a token the model conditions on, not a label handed back from an external index. Retrieval-by-cosine returns a neighbour. Our memory returns something the model can think with, in constant time and with no per-user training. On raw recall the two are close, and a metric learned on the encoder closes what gap remains. The rest of the paper is about the behaviour that survives

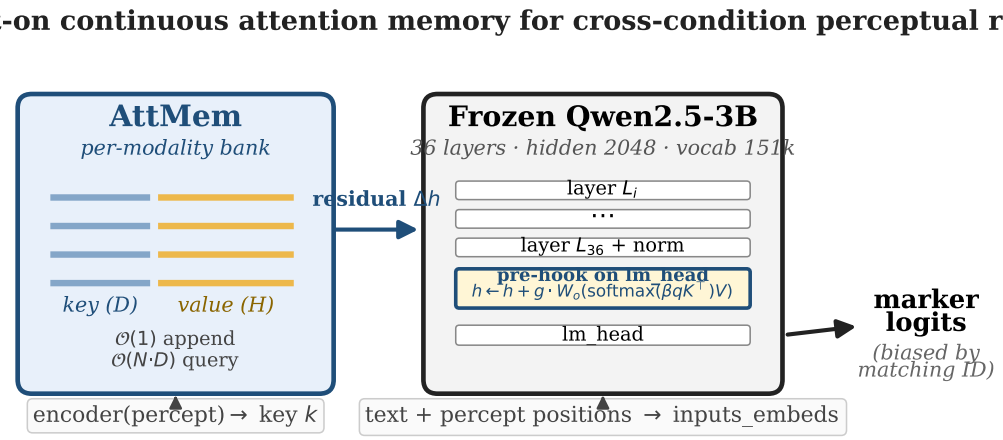


Figure 2. A frozen language model is augmented by a forward pre-hook at its output head. Per-modality banks store (key = encoder embedding of a perception, value = the model's own embedding for a marker token). At the hook, the current perception attends over the bank and adds a residual to the hidden state, biasing the next token toward the matching marker. Registration is a single tensor append. About 8M trainable parameters sit on top of 3.1B frozen.

that comparison: the in-model recall a retrieval index cannot give you.

3 The mechanism: a perception as a row in a bank

The whole memory is a table of rows, one per registered perception (Figure 2). Row i holds a *key* $\boldsymbol{k}_i$ and a *value* $\boldsymbol{v}_i$. The key is the L2-normalized embedding the modality's encoder produces for the perception (a face, a voiceprint, a style vector). The value is the language model's own embedding for a marker token assigned to that identity. Storing the model's native vector as the value, rather than an arbitrary learned one, is deliberate. It makes the eventual logit boost for the right marker clean and self-reinforcing, rather than an accident of two unrelated vectors.

Reading the memory is one step of attention. When a perception arrives at generation time, the model forms a query $\boldsymbol{q}$ from it and, just before the output head, computes

$$\boldsymbol{w} = \mathrm{softmax}\big(\beta\, \boldsymbol{q}^\top \boldsymbol{K}\big), \qquad \boldsymbol{r} = \boldsymbol{w}^\top \boldsymbol{V}, \qquad \boldsymbol{h}' = \boldsymbol{h} + g \cdot W_o\, \boldsymbol{r},$$

where $\boldsymbol{K}, \boldsymbol{V}$ stack the bank's keys and values, β is a learned sharpness, g a learned gain, and W_o a learned projection (initialized to identity). In words: the perception softly looks up the bank, pulls back a blend of the matching markers' model-side vectors, and nudges the next-token prediction toward them. The blend is the entire mechanism. It adds about 8M trainable parameters over the frozen model.

Getting it to work was four bug-fixes, not four ideas. The architecture above is the fourth iteration. Three earlier versions produced random output despite healthy-looking loss curves. The fixes are unglamorous, and worth stating because they are where the time went. (i) *Do not* divide the attention logits by $\sqrt{D}$. With normalized keys, that flattens every cosine difference into near-uniform attention. (ii) Initialize the sharpness β high, so attention is decisive from

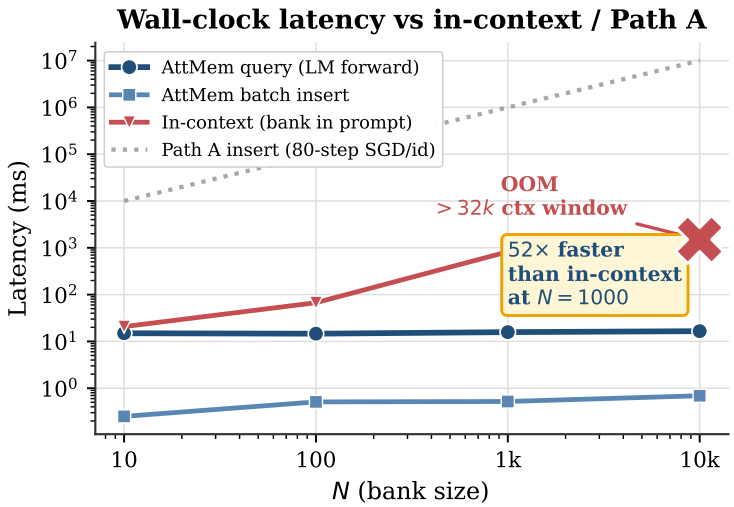


Figure 3. Recall latency is flat in bank size (∼15 ms, dominated by the frozen model's own forward pass) and insertion is $\mathcal{O}(1)$. Feeding the same registered perceptions in as context grows linearly and exhausts the context window past ten thousand identities. Per-concept fine-tuning (PATH A-style) costs gradient steps per identity. Log–log axes.

the first step. (iii) Attach the hook at the output head, not at an intermediate layer, so the residual reaches the logits undiluted. (iv) Give the gain a large learnable value, so the nudge actually overcomes the model's built-in reluctance to emit unusual marker tokens. A fifth choice, varying the bank size during training, is not a correctness fix but matters for scaling to large memories, and we return to it in Section 6. Training the memory on a modality takes a few thousand steps. Once trained, individual users are registered with no training at all.

Why register, rather than fine-tune. The economics are the point (Figure 3). Adding an identity is a single `torch.cat` (about half a millisecond to add a thousand), and recall is constant-time regardless of bank size, because the lookup is a tiny matrix multiply dwarfed by the model's own forward pass (∼15 ms either way). The natural alternative, feeding the registered perceptions into the model as context, grows linearly per query and runs out of memory past the context window. Our mechanism is 52× faster at a thousand identities, and is the only one of the two that still functions at ten thousand. Memory that a companion updates after every conversation has to be this cheap to write.

4 The PerceptMem benchmark

To measure perceptual recall honestly we built **PerceptMem**, five sub-modalities each chosen for one reason: the signal that discriminates is provably destroyed by captioning (Table 2). They span vision and audio, identity and style, the physical and the affective:

The sources are all standard: LFW [Huang et al., 2008] and AgeDB [Moschoglou et al., 2017] read with ArcFace [Deng et al., 2019], WikiArt [WikiArt, 2010] with mid-layer CLIP [Radford et al., 2021], LibriSpeech [Panayotov et al., 2015] with ECAPA-TDNN [Desplanques et al., 2020], ESC-50 [Piczak, 2015] with AST [Gong et al., 2021], and RAVDESS [Livingstone and Russo, 2018] with a wav2vec2 emotion encoder [Baevski et al., 2020]. The benchmark deliberately tests *memory*, not perception: every sub-modality has a fixed, off-the-shelf encoder that any

Table 2. The five PerceptMem sub-modalities. Each is included because no caption can carry its discriminating signal.

Sub-modality	Source / encoder	Why text cannot carry it
Face across age & lighting	LFW+AgeDB / ArcFace	"brown-haired man" fits thousands
Painter style	WikiArt / CLIP-mid	no caption separates early vs. late Monet
Speaker across recordings	LibriSpeech / ECAPA-TDNN	timbre is not transcribable
Acoustic scene	ESC-50 / AST	"traffic noise" fits every street
Tone of voice	RAVDESS / wav2vec2	"sounded tired" lacks a personal baseline

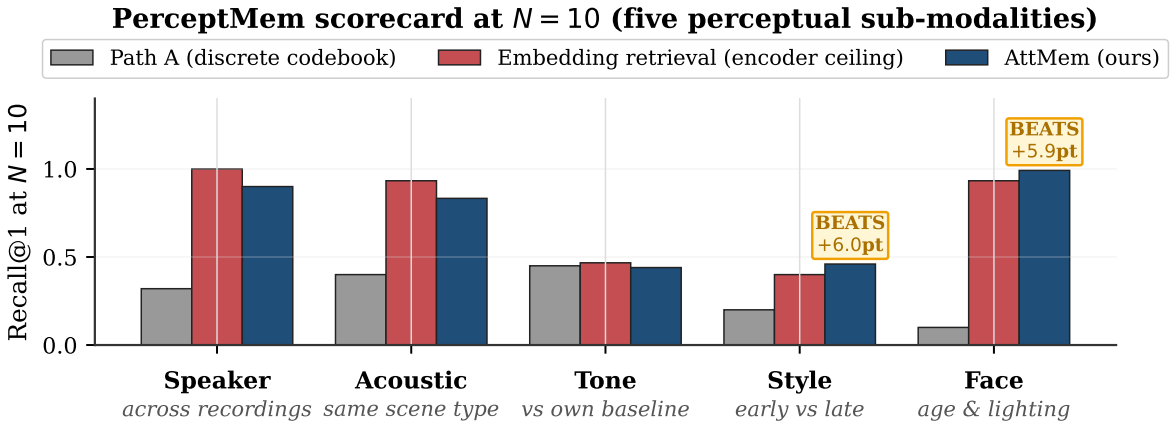


Figure 4. Recall at ten registered identities across the five sub-modalities. Parametric memory (navy) *exceeds* embedding retrieval on face and style, *ties* it on tone of voice, and trails by about ten points on acoustic scene and speaker. Those are the two where the encoder is already near-perfect and there is nothing left to add. The discrete-codebook predecessor (PATH A, grey) is several times lower everywhere.

method may use as black-box infrastructure, so no one wins by having a better encoder. The interface is just `register` and `recall`. Data is cross-session: a registered sample and its query come from different recordings. The identities used to train the memory never overlap with those it is evaluated on. For a memory of N identities we register one sample each, then issue cross-condition queries and ask the method to name the right one. For tone of voice, each registered "identity" is a (speaker, emotional-state) pair, so recalling it means matching a paralinguistic state across separate utterances, something a per-utterance caption cannot do without the user's own history.

Throughout, our reference to beat is **embedding retrieval**: the same encoder, cosine nearest-neighbour over the registered keys. We sometimes call it the *encoder ceiling*, because it is the best one can do with the encoder's own similarity. But it is not a ceiling for our method, which measures similarity in the model's representation space instead, and can therefore exceed it.

5 What the memory can do

The headline is simple. Where the encoder's own cosine similarity is imperfect, a learned re-encoding helps, and our memory beats raw retrieval (and matches a learned metric on the same encoder). Where the encoder is already perfect, there is nothing to add and our memory ties. Figure 4 shows this across all five sub-modalities at a memory of ten.

It edges raw retrieval, but the recall gain is metric learning. On a 2180-identity face pool, raw embedding retrieval reaches 0.93 recall at ten identities and our memory reaches 0.99 (p=0.006 across four seeds). On painter style, retrieval over mid-layer CLIP features manages 0.40 at five painters and our memory reaches 0.64. These are real gains over *raw* cosine, but they are not unique to the parametric memory, and intellectual honesty requires us to say so. A similarity learned on the same encoder and the same training identities captures the same lift. A closed-form LDA reaches 0.97 on faces at ten identities, and a within-class whitening reaches 0.60 on style at five painters, both well above raw cosine and on par with our memory (Appendix A). The recall surplus over raw cosine is therefore metric learning rather than a property of reading in the model's representation space. And at large pools no re-encoding wins. Past a few hundred identities raw cosine beats our memory and the learned metrics alike, because the bottleneck is the encoder's cross-condition invariance, which none of them change.

What is distinct is that the memory lives in the model. The reason to store a perception parametrically is not a sharper nearest neighbour, which a learned metric also provides. It is that the bank is read by attention at the output head, so a recalled identity is a *token the model can condition its generation on*, not a label returned by an external index. Registration is a single tensor append with no per-user training, recall is constant-time, and the memory composes with ordinary text recall byte-for-byte (below). A learned retrieval metric gives the agent a better neighbour. It does not give the agent a memory it can think with. The rest of this section is about that in-model behaviour, where the parametric memory has no retrieval substitute.

It holds up against look-alikes. The face pool tests random distractors. The harder question is what happens when the other identities in memory are the ones most easily confused with the target, such as siblings in a family album, or several recordings from the same small group. By construction this is where the encoder's similarity is most confused, and where plain training has the least to add: standard-trained memory actually trails retrieval here. The fix is to *train against the confusion*, mixing hard look-alike banks into a fraction of training. This transforms the regime (Figure 5). On tone of voice, recall against nineteen look-alikes climbs from 0.23 to 0.93. On painter style, it climbs from 0.27 to 0.98. On acoustic scene it becomes perfect, 1.000 across every seed. On faces it gains fourteen points. The largest wins land exactly where the encoder was weakest. The memory turns the encoder's blind spot into the model's advantage.

It is content-addressable, and you can see it work. Figure 6 opens up a single recall. The encoder's cosine matrix (b) is diagonal-dominant but speckled with off-diagonal mass, and those speckles are retrieval's errors. Read in the model's representation space, the *same* keys produce a far sharper attention map (a): its diagonal averages 0.98 against the cosine's 0.46. That gap is the surplus signal, and it carries through to the marker logits (c), whose argmax lands on the right identity. The sharpening comes from the model's representation, not from any extra view of the input.

It composes with text memory, for free. A parametric perceptual memory is meant to sit beside an ordinary text memory, not replace it. We checked that it does no harm: register faces

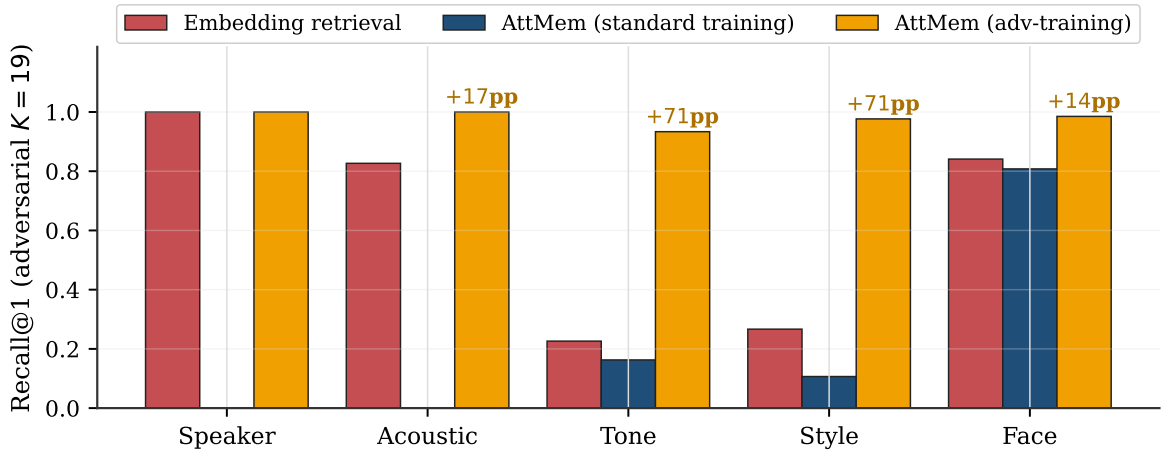


Figure 5. Against the nineteen most-confusable distractors, training the memory to expect look-alikes beats embedding retrieval by +14 to +71 points on four of five sub-modalities. The fifth (speaker) has no headroom, because its encoder is already perfect. The biggest gains are on the sub-modalities where the encoder's similarity was weakest to begin with.

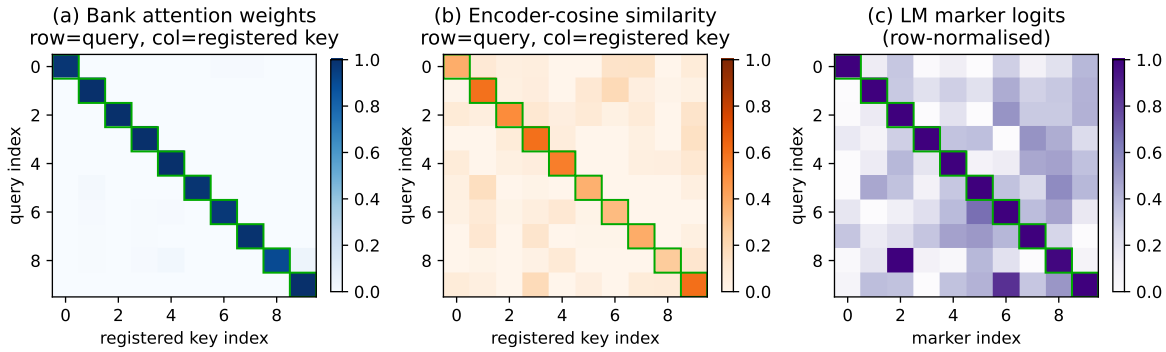


Figure 6. One recall, three views (held-out faces, ten identities). **(a)** read in the model's space, bank attention is a near-pure diagonal (avg. 0.98), so each query locks onto its target. **(b)** the *same* keys under encoder cosine give a much softer diagonal (avg. 0.46) with off-diagonal mass, and this is where retrieval slips. **(c)** the resulting marker logits keep the correct identity on top. The (a)-vs-(b) gap is the "sharper ruler."

and voices together and the two banks stay independent without any training (cross-modal leakage 0.017 and 0.067, zero-shot), and on plain propositional prompts the model's next-token predictions are *byte-for-byte identical* to the untouched model. The perceptual memory fires only at perceptual moments. Everything else passes through unchanged. An agent can keep "my favourite restaurant is in Paris" in its text store and "how Bibi looks across lighting and age" in its perceptual bank, and lose nothing on either.

Why not just quantize the perception into a code? We spent sixteen development cycles on exactly that. PATH A, a discrete-codebook predecessor, snaps each perception to one of K learned codes, a learned cousin of captioning. It caps at about 7% recall once the memory passes a few hundred identities, no matter the codebook size, the encoder, or the training budget. That is roughly a 9× gap from continuous attention at scale (Figures 1, 11b). The reason is the same one that sinks captioning: two perceptions that land in the same code are

indistinguishable forever after. Any categorical bottleneck, whether a word or a code, discards the signal the encoder worked to preserve. Keeping the perception continuous is the whole game.

6 When it helps, and when it doesn't

An honest account has to say where the mechanism adds nothing, and why. Three regimes cover it.

When the encoder is imperfect, training adds real signal. On the face pool, an untrained memory is below retrieval at every size. Training pulls it above, and the lift *grows* with the memory, from a few points at ten identities to forty points at seven hundred. The memory is learning genuine per-modality discrimination, not just inheriting the encoder's cosine.

When the encoder is already perfect, training only adds noise. On speaker identity the encoder hits 1.00 recall on its own. An untrained memory matches it exactly, and a *trained* one is slightly worse, because the learned projection perturbs a signal that needed no help. When cosine is already at ceiling, the model has nothing to contribute, and our scorecard says so plainly rather than hiding it.

A structural rule, from putting it inside a real VLM. The cleanest finding came from running the memory inside a vision-language model (Qwen2.5-VL) that sees raw face images through its own visual front-end. With an external face encoder (ArcFace) as the key, the memory reproduces its win, with perfect recall at ten identities. With the VLM's *own* vision tokens as the key, it cannot beat retrieval at all. The difference is geometric: the VLM's vision tokens already live in the model's hidden space, so the key and the value point the same way and the model's representation offers no *new* direction to discriminate along. The lesson is a design rule: **the memory's key must be orthogonal to the model's value space for the second ruler to be sharper than the first**. It explains why a purpose-built encoder (ArcFace for faces, ECAPA for voices, CLIP for style) beats a general-purpose one here.

It travels across model families, but scale is not the lever. The mechanism is not tied to one model. It transfers from Qwen2.5-3B to 7B, and to Llama-3.1-8B, which on the same recipe beats retrieval and edges out Qwen-3B at large memories. It needs only an automatic adjustment for whether a model ties its input and output embeddings. Within a family, though, size does not help: at a fixed training budget Qwen-7B is no better than Qwen-3B at large memories, because the bottleneck is the encoder's cross-condition invariance and the gradient signal at the bank's softmax, neither of which scales with the frozen model. The one place a bigger model pays off is the look-alike regime, where its richer representation recovers hard cases while keeping easy ones. Portability is real but recipe-sensitive. Mistral-7B did not converge in the same budget, which we report rather than paper over.

7 Discussion

The claim is structural, and the numbers only illustrate it. The argument is not "our method scores higher." It is that text is the wrong container for half of what an agent should remember about a person. The empirical wins matter because they show the alternative container is not merely lossless but actively sharper, and that this surplus appears only where the encoder's similarity left discrimination on the table. That boundary is the honest shape of the

contribution.

This is additive infrastructure, not a replacement. The perceptual memory is designed to bolt onto the text memory that agents already have [Chhikara et al., 2025, Wang et al., 2024, Packer et al., 2023, Team, 2024], and the non-regression result is what makes that safe: text recall is preserved exactly, and the perceptual bank activates only at perceptual moments. As long-term memory benchmarks like LongMemEval [Wu et al., 2024] grow to include perceptual content, the gap this paper argues for becomes directly measurable.

The captionable half is going parametric too. This paper is one half of a larger bet: a personalized agent should keep its memory *inside* the model, not in an external index. A companion line makes the same bet for the *captionable* half. *User as Engram* [Li, 2026c] writes per-user *facts* into a hash-keyed parametric memory, showing that the obvious alternative (a per-user LoRA adapter) is *reasoning-negative*: it memorizes facts but degrades the reasoning that uses them, because a LoRA edit is global where a row insertion is local. *User as Code* [Li, 2026b] pushes the same half into executable typed state, and *Programmable KV Cache* [Li, 2026a] keeps it as editable, composable notes in the attention cache rather than in weights or an external index. Our perceptual counterpart shares that thesis with a different mechanism, continuous cross-attention over encoder banks rather than hash-keyed rows. The two halves compose into one memory that holds both what the user said and what they are like.

What a latent memory can and cannot hold. The split between the two stores is not a convention. It follows from two capacity laws that we measured by compressing content into k soft tokens read by a frozen language model (Figure 7). Perceptual identity is capacity-limited and degrades gracefully. Compressing M registered faces or voices into k prototype slots and recognising a cross-condition query gives recall close to $\min(1, k/M)$. One slot per identity recognises everyone, and fewer slots lose identities in proportion. This law is not an artifact of k-means: a compressor whose k slots we instead learn by gradient descent lands on the same $\min(1, k/M)$ curve, because k hard slots can keep at most k of M identities separable (a pigeonhole bound). The only way past it is to attach a per-identity soft code over the slots, which restores accuracy but costs storage linear in M, at which point one may as well keep the M keys. Exact factual content behaves in the opposite way. A latent holds one exact code and reads it back (0.87 for a six-character code), but storing several and retrieving one by name collapses at once (0.06 at two codes, near zero beyond), and adding latent tokens does not help, with recall pinned at the floor from sixteen to a hundred and twenty-eight tokens. Recognition is a capacity problem, where a latent buys one identity per slot. Exact recall is a binding problem, where a latent cannot match a key to its value at any budget. That is why the captionable half wants a text store, which performs that lookup exactly, and the perceptual half wants a parametric bank, which scales with its slots.

Limitations. (i) At very large memories the encoder's own ceiling reasserts itself. Our memory reaches 0.59 at a thousand faces against the encoder's 0.77, and further training yields diminishing returns. (ii) We have measured real cross-condition data to roughly two thousand identities. The constant-time behaviour is confirmed to ten thousand, but recall at that scale awaits data. (iii) Inside a VLM, native vision tokens only tie retrieval and stay below an external encoder, so an external, modality-specific encoder is preferred. (iv) Portability is recipe-sensitive (Mistral-7B). (v) The closest prior systems (MyVLM [Alaluf et al., 2024], Online-PVLM [Bai et al., 2025]) have released neither code nor checkpoints. We compare against charitable reimplementations of their core mechanisms (Appendix A, Table 5); a

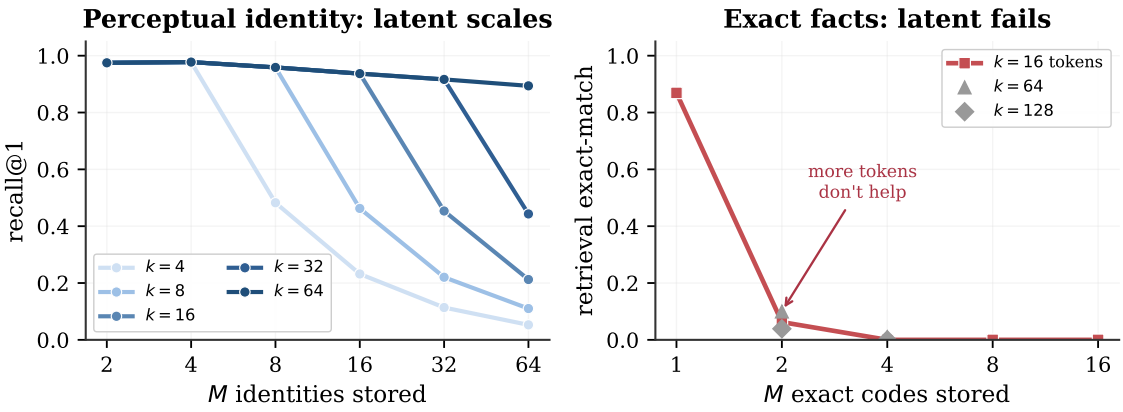


Figure 7. Two capacity laws for memory compressed into k soft tokens read by a frozen language model, the mechanism behind the router. **Left:** perceptual identity is capacity-limited. Compressing M faces into k prototype slots and recognising a cross-condition query gives recall $\approx \min(1, k/M)$, so a slot per identity recognises everyone and more slots hold more people (voices behave identically). **Right:** exact factual content is binding-limited. A latent holds one exact code (0.87) but multi-code retrieval collapses at two (0.06) and adding latent tokens does not rescue it. Recognition scales with the latent budget. Exact recall does not, so it belongs in a text store.

faithful head-to-head against their full systems awaits release.

8 Conclusion

A photograph and a written description of a face are not two encodings of the same thing. The description has already thrown away what lets you recognize the person. Agent memory today keeps only the description. We have argued it must keep the photograph too. The perception is stored in its native modality, as a content-addressable row in a bank on a frozen language model, not flattened into words it cannot survive. The two memories are complementary: text for what the user *said*, parametric multimodal memory for what the user *is like*. An agent with both strictly dominates one with only text, recalling the captionable half exactly and holding the perceptual half at all.

Empirically, keeping the perception continuous beats raw retrieval wherever the encoder's cosine similarity has room to improve, by up to 24 points on random banks and 14 to 71 once the memory is trained to expect look-alikes. A metric learned on the same encoder reaches the random-bank gains too, so the recall number is not the contribution. What a retrieval index cannot copy is the rest: the recalled perception is a token inside the model, registered in constant time, composing with text recall, and the memory is cheap enough to update after every conversation and portable across model families. That is where a face stops being a sentence, and starts being a memory.

Acknowledgements

AI tools including Pine Copilot, Claude Code with Claude Opus 4.8 were used during this research. We thank BSQL Networking for hosting the RTX PRO 6000 GPU.

A Detailed results

This appendix collects the per-cell numbers behind the narrative in Sections 5–6. Unless noted, recall is top-1 with the argmax restricted to registered markers. "±" is sample standard deviation (ddof= 1) across seeds, and each p-value is a two-sided one-sample t-test of the seed-level AttMem recalls against the deterministic retrieval value, with n as listed. The seven significant cells below are those that clear $p<0.05$ in the full sub-modality×size sweep. We report each per-cell and apply no family-wise correction, so the p-values should be read individually.

Table 3. The seven cells where parametric memory beats embedding retrieval at $p<0.05$ (multi-seed). "Random" banks draw distractors uniformly. "Adversarial" banks use the K most cosine-similar distractors and adversarially-aware training.

Regime	Cell	n	Retrieval	AttMem (mean±std)	Δ	p
random	Face, N=10	4	0.933	0.992 ± 0.017	+5.9pp	0.006
random	Style, N=5	5	0.400	0.640 ± 0.130	+24pp	0.015
random	Style, N=10	5	0.400	0.460 ± 0.028	+6pp	0.009
adversarial	Face, K=19	3	0.841	0.985 ± 0.001	+14.4pp	$< .001$
adversarial	Acoustic scene, K=19	4	0.827	1.000 ± 0.000	+17.3pp	$< .001$
adversarial	Tone of voice, K=19	4	0.226	0.934 ± 0.005	+70.7pp	$< .001$
adversarial	Style, K=19	4	0.267	0.977 ± 0.007	+71.0pp	$< .001$

Learned-metric baseline. The AttMem recalls above are gains over *raw* cosine. To test whether they are simply metric learning, we fit a similarity on the same encoder features and the same training identities (the 50/50 identity split, seed 42), then score it with the identical recall@1 protocol. Two closed-form metrics suffice: regularised LDA and within-class whitening. Table 4 shows a learned metric matches or beats AttMem's small-memory recall on both modalities where AttMem claims a random-bank win, and that past a few hundred identities raw cosine beats AttMem, LDA, and whitening alike. The recall surplus over raw cosine is therefore attributable to metric learning, not to reading in the model's representation space. We report this rather than omit it: AttMem's defensible contribution is the in-model behaviour (Sections 5 and 7), which no retrieval metric provides, not the recall@1 number. A contrastive linear head we also trained underperformed both closed-form metrics and is omitted as a weak instantiation.

Per-concept methods: a charitable head-to-head. The personalized-VLM line (MyVLM, Yo'LLaVA, Online-PVLM) registers a concept by *learning* something per concept (a classifier head, a soft token, a projection). Their code and checkpoints are unreleased, so we reimplement each method's core mechanism charitably on the cross-condition face task (ArcFace keys, same split). Table 5 reports accuracy and registration cost. Two findings. First, a per-concept linear classifier (MyVLM's mechanism) is statistically indistinguishable from raw cosine at every memory size, because with one registration embedding per identity the trained head collapses to the embedding itself. It buys no accuracy over retrieval, and it costs per-identity gradient descent that grows to 4.5 s at a thousand identities, against our $\mathcal{O}(1)$ tensor append. Second, a learned projection trained with supervised contrastive loss (Online-PVLM's mechanism)

Table 4. Learned-metric baseline, recall@1, same encoder and same split as AttMem. A learned metric (LDA on faces, whitening on style) matches AttMem's small-N lift over raw cosine; at large N raw cosine wins for all. Style AttMem here is the seed-42 single run (the multi-seed mean is 0.64 at N=5); the comparison is within seed.

Modality	N	Raw cosine	LDA	Whitening	AttMem
Face (ArcFace)	5	0.933	**1.000**	0.933	0.933
Face (ArcFace)	10	0.933	0.967	0.933	**1.000**
Face (ArcFace)	300	0.734	0.667	**0.744**	0.641
Face (ArcFace)	1000	0.767	0.724	**0.776**	—
Style (CLIP-mid)	5	0.400	0.200	**0.600**	0.467
Style (CLIP-mid)	10	0.400	0.333	0.367	**0.467**

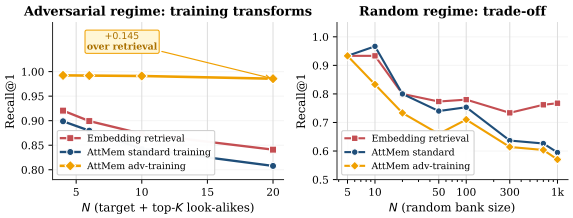
underperforms raw cosine by 15 to 28 points on this task. We read this as a caution that a learned re-encoding is not free: tuned on identity-disjoint data, it can distort the very cross-condition geometry it was meant to sharpen. This may also reflect our reimplementation rather than the full system, which we cannot run. The honest summary is that no method here, ours included, beats raw cosine on accuracy at scale, and the parametric memory's case rests on cost and in-model behaviour, not recall.

Table 5. Charitable reimplementations of per-concept methods on cross-condition face recall (ArcFace, recall@1, single eval draw). MyVLM's per-concept classifier ties raw cosine but its registration cost grows with the memory; Online-PVLM's learned projection underperforms raw cosine. AttMem leads only at small N and trails at scale (the encoder-ceiling inversion).

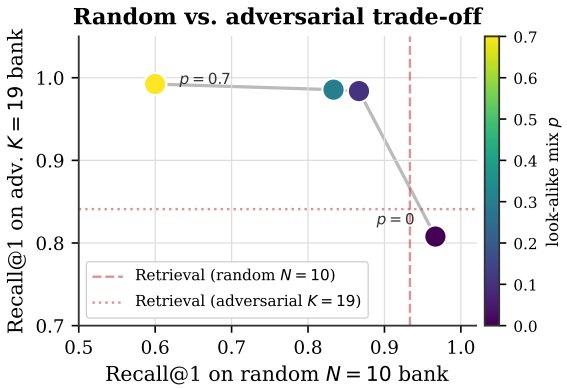
N	Raw cosine	MyVLM	Online-PVLM	AttMem	MyVLM reg. cost
5	0.933	0.933	0.800	0.933	0.009 s
10	1.000	0.967	0.750	0.992	0.019 s
50	0.767	0.753	0.675	0.733	0.178 s
100	0.780	0.770	—	0.742	0.528 s
300	0.728	0.725	—	0.637	1.39 s
1000	0.776	0.778	—	0.594	4.52 s

The look-alike regime in detail. Standard training (no look-alike exposure) trails retrieval on the confusable banks where the encoder has headroom: −3.3pp on faces, −6.3pp on tone of voice, and −16.0pp on style. It ties on speaker (both 1.00). Adversarially-aware training (mixing hard banks into 30% of steps) reverses this completely, giving the Table 3 adversarial rows. The random and adversarial regimes trade off, and the shape of the trade-off is modality-dependent: on faces the frontier is gradual (a 10% look-alike mix keeps 0.87 random while reaching 0.98 adversarial), whereas on tone of voice any look-alike mix sharply lowers random recall while adversarial saturates at once. A 10% mix is the sweet spot for random-population deployments. A 30% mix suits known-adversarial settings (siblings, same-room recordings). A larger model family recovers both at once (Figures 8–9).

Scaling, training, and ablations. Figure 10a traces recall against memory size on the face pool: parametric memory beats retrieval at ten identities and tracks within ∼17 points through



(a) standard vs. look-alike training (faces)



(b) random/adversarial trade-off (faces)

Figure 8. Training the memory to expect look-alikes transforms the adversarial regime at some cost to random-bank recall. A ∼10% mix is the sweet spot on faces, and the curve's shape varies by modality.

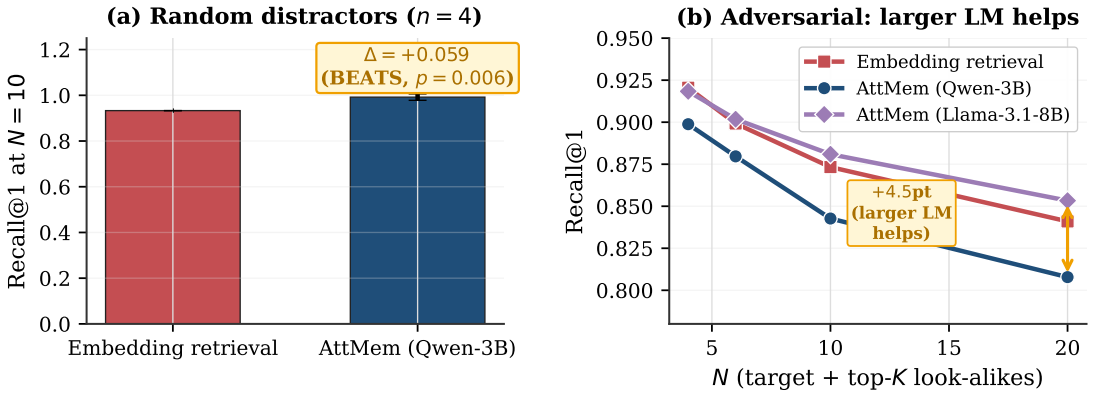
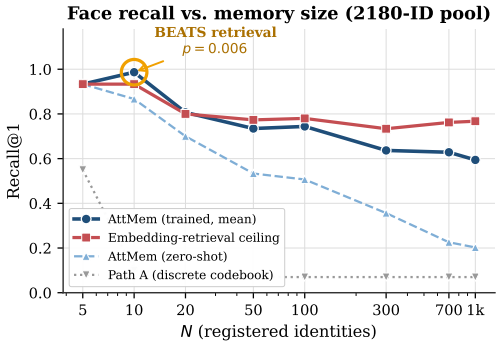


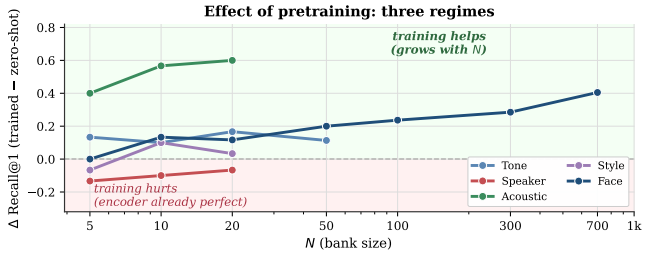
Figure 9. Random vs. adversarial distractors. (a) on random banks, parametric memory beats retrieval by 5.9 points (p=0.006). (b) on adversarial banks, a small frozen model (Qwen-3B) trails retrieval by a few points under standard training, while a larger one (Llama-3.1-8B) matches or beats it even without look-alike training. The larger model has more representational room to discriminate when the encoder is confused.

a thousand, while the discrete-codebook predecessor flatlines at ∼0.07. Figure 10b decomposes the effect of training per sub-modality (the three regimes of Section 6). Figure 11a reports two ablations: at fixed budget a larger frozen model is not better at large memories, and varying the bank size during training is essential. Fixed-size training collapses from 0.63 to 0.20 recall at seven hundred identities because of a train/test size mismatch. Figure 11b contrasts the discrete codebook against continuous attention across all five sub-modalities.

Vision-language model, key/value orthogonality. Table 6 gives the numbers behind the structural rule of Section 6. Each memory row is compared to retrieval *in its own key space*. An external ArcFace key (orthogonal to the model's hidden space) reproduces the win. The VLM's native vision tokens (already in hidden space) only tie native-token retrieval and stay far below the ArcFace ceiling throughout, so they are the wrong key regardless of method.

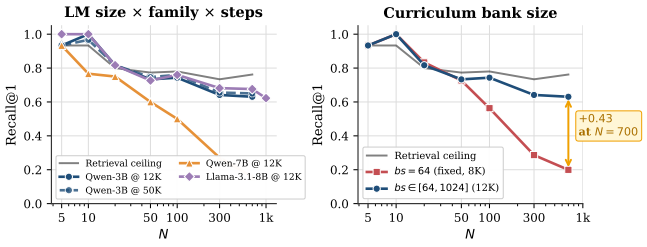


(a) scaling on the face pool

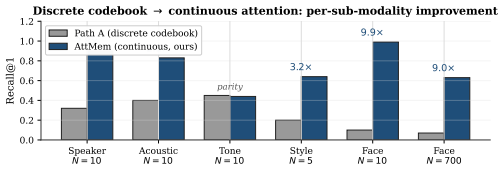


(b) effect of training, per sub-modality

Figure 10. Left: parametric memory beats retrieval at small memories and degrades gracefully, while the codebook predecessor never gets off the floor. Right: training helps most where the encoder is imperfect, and slightly hurts where it is already perfect.



(a) model size and curriculum bank size



(b) codebook vs. continuous attention

Figure 11. Left: within a fixed budget a bigger frozen model does not help at scale, and varying the bank size during training is what makes large memories work. Right: replacing the discrete codebook with continuous attention lifts recall 2–10× per sub-modality.

Table 6. Parametric memory inside Qwen2.5-VL-3B: same frozen model, only the key encoder changes. Recall at three memory sizes.

Configuration	N=10	N=100	N=1000
Retrieval over ArcFace embeddings (encoder ceiling)	0.933	0.780	0.767
AttMem + external ArcFace keys (orthogonal to hidden space)	**1.000**	0.710	0.494
Retrieval over native vision tokens	0.40	0.35	—
AttMem + native vision tokens (co-located with hidden space)	0.40	0.11	—

B Anatomy of a single recall

It helps to watch one recall end to end. Suppose the agent has met ten people and wants to register an eleventh from a single photo. Registration is one row appended to the face bank (Listing 1). The encoder turns the photo into a 512-dimensional ArcFace embedding, the *key*. The model's own embedding for a freshly assigned marker token becomes the *value*, a vector of the model's hidden width (2048 for Qwen2.5-3B). There is no gradient step and no fine-tuning. The row is the memory.

At recall, a new photo of one of the eleven arrives under different lighting and a year older. Its ArcFace embedding becomes the query q. Just before its output head, the model scores q against all eleven keys, softmaxes into attention weights w, and pulls back the weighted blend

```
# Register identity #11 from a single photo -- no training:
k = l2_normalize(arcface(photo))        # key:   R^512  (encoder space)
m = assign_marker_token()               # e.g. "<id_11>"
v = model.input_embedding[m]            # value: R^2048 (model's own space)
bank.K = torch.cat([bank.K, k[None]])   # O(1) append
bank.V = torch.cat([bank.V, v[None]])
```

Listing 1. One registered identity is one row. The key is the encoder's view of the perception; the value is the model's own vector for the marker token that names it. Adding the row is a single tensor append.

of values $\boldsymbol{r} = \boldsymbol{w}^\top \boldsymbol{V}$. Because each value is the model's own vector for a marker token, adding $g\, W_o \boldsymbol{r}$ to the hidden state lands as a clean boost on the right marker's logit. The model's next token *is* the remembered identity. The whole step is a few matrix multiplies dwarfed by the model's forward pass.

The 0.98-vs-0.46 diagonal gap of Section 5 (Figure 6) is measured on exactly this probe: a held-out ten-identity bank, with the same encoder and keys used at registration, reading attention in the model's space versus the encoder's cosine. That gap is the "sharper ruler" in miniature: one query, one bank, the surplus signal made visible.

C Why the quantized predecessor capped at 7%

Before continuous attention we spent sixteen development cycles on PATH A: a discrete codebook that snaps each perception to one of K learned codes and remembers the code. It is the natural "compress the perception into a symbol" design, and it is a learned cousin of captioning, which is why its failure is instructive rather than embarrassing.

No setting broke it. Across codebook sizes $K \in \{128, 256, 512, 1024\}$, even after *100,000* steps of continual pretraining on the expanded face pool, top-1 recall at 300 identities stays pinned between 0.057 and 0.070 while embedding retrieval over the same encoder reaches 0.73. That is a roughly $10\times$ gap that does not move with K (Table 7). Swapping ArcFace R50 for AntelopeV2 R100 trained on 360K identities did not help either.

Table 7. PATH A after 100K-step continual pretraining on the 2180-identity face pool, at 300 registered identities. Recall is pinned near 0.07 regardless of codebook size, even when the gate routes to the correct code about half the time.

Codebook size K	128	256	512	1024
PATH A recall@1	0.057	0.064	0.070	0.070
gate routes to correct code	0.43	0.51	0.42	0.54
embedding retrieval (same encoder)		0.73		

The reason is a vice the codebook cannot escape, and the diagnostics name it exactly. Two pressures pull in opposite directions as K grows. A *small* codebook packs many people into each cell: at K=16, distinct identities collide in the same code 8.7% of the time, and once two people share a code they are indistinguishable forever after. A *large* codebook fixes that, with inter-identity collisions falling to 2.4% at K=128, but it shatters each identity across cells. The rate at which two cross-condition photos of the *same* person land in the same code drops from 0.33 to 0.20, so the query no longer routes to where the registration was stored. Net recall is

squeezed from both sides and never escapes ∼0.07. Even when the gate does route a query to the right code (about half the time, Table 7), every other identity sharing that cell is an equally good answer, so the final pick is near-random.

This is the same information loss as captioning, learned instead of written: any categorical bottleneck, whether a word or a code, throws away the continuous signal the encoder worked to preserve, and no amount of compute downstream can recover it. Continuous attention over the raw embedding never quantizes, and never pays this tax. PATH A is in the paper because its failure is the cleanest possible argument for the design that replaced it.

D Reproducibility

The system is about 200 lines of new code over a frozen-model `transformers` stack. The training/eval entry point takes `mode, n_steps, seed, [bank_size_max], [adv_prob]`. Every run logs to `results/`, and aggregation plus the multi-seed *t*-tests are in the repository. A face run is ∼17 minutes on an H100-class GPU (12K curriculum steps). Audio and style runs are ∼5–10 minutes (5K steps).

```
# Random-regime wins (multi-seed, p<0.05)
for s in 42 43 44 47; do
  python3 attmem_train_and_eval.py v-xc-id-xxxl 12000 $s 1024
done
for s in 42 43 44 45 46; do
  python3 attmem_train_and_eval.py v-sty-clip 5000 $s
done

# Look-alike regime (multi-seed, p<0.001)
for s in 49 50 51; do
  python3 attmem_train_and_eval.py v-xc-id-xxxl 12000 $s 1024 0.3
done
for s in 42 43 44 45; do
  python3 attmem_train_and_eval.py a-para     5000 $s 0 0.3
  python3 attmem_train_and_eval.py v-sty-clip 5000 $s 0 0.3
  python3 attmem_train_and_eval.py a-scn      5000 $s 0 0.3
done

# Cross-family, VLM, latency, text non-regression
ATTMEM_MODEL_ID="NousResearch/Meta-Llama-3.1-8B-Instruct" \
  python3 attmem_train_and_eval.py v-xc-id-xxxl 12000 42 1024 0.3
python3 attmem_vl_train.py 3000 42 0
python3 attmem_latency_benchmark.py
python3 attmem_propositional_control.py
```

References

Yuval Alaluf, Elad Richardson, Sergey Tulyakov, Kfir Aberman, and Daniel Cohen-Or. MyVLM: Personalizing VLMs for user-specific queries. In *European Conference on Computer Vision (ECCV)*, 2024.

Ruichuan An, Sihan Yang, Ming Lu, Kai Zeng, Yulin Luo, Ying Chen, Jiajun Yong, Huanqia

Liang, Qi Huang, and Shanghang Zhang. MC-LLaVA: Multi-concept personalized vision-language model. *arXiv preprint arXiv:2411.11706*, 2024.

Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in Neural Information Processing Systems*, 2020.

Huiyu Bai, Runze Wang, Zhuoyun Du, Yiyang Zhao, Fengji Zhang, Haoyu Chen, Xiaoyong Zhu, Bo Zheng, and Xuejiao Zhao. Online-PVLM: Advancing personalized VLMs with online concept learning. *arXiv preprint arXiv:2511.20056*, 2025.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. ArcFace: Additive angular margin loss for deep face recognition. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

Brecht Desplanques, Jenthe Thienpondt, and Kris Demuynck. ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification. In *INTERSPEECH*, 2020.

Yuan Gong, Yu-An Chung, and James Glass. AST: Audio spectrogram transformer. In *INTERSPEECH*, 2021.

Haoran Hao, Jiaming Han, Changsheng Li, Yu-Feng Li, and Xiangyu Yue. RAP: Retrieval-augmented personalization for multimodal large language models. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

Gary B Huang, Marwan Mattar, Tamara Berg, and Eric Learned-Miller. Labeled faces in the wild: A database for studying face recognition in unconstrained environments. In *Workshop on Faces in Real-Life Images*, 2008.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations (ICLR)*, 2020.

Bojie Li. Models take notes at prefill: KV cache can be editable and composable. *arXiv preprint arXiv:2606.17107*, 2026a.

Bojie Li. User as code: Executable memory for personalized agents. *arXiv preprint arXiv:2606.16707*, 2026b.

Bojie Li. User as engram: Internalizing per-user memory as local parametric edits. *arXiv preprint arXiv:2606.19172*, 2026c.

Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning (ICML)*, 2023.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Steven R Livingstone and Frank A Russo. The Ryerson audio-visual database of emotional speech and song (RAVDESS). volume 13, 2018.

Lin Long, Yichen He, Wentao Ye, Yiyuan Pan, Yuan Lin, Hang Li, Junbo Zhao, and Wei Li. Seeing, listening, remembering, and reasoning: A multimodal agent with long-term memory (M3-Agent). *arXiv preprint arXiv:2508.09736*, 2025.

Stylianos Moschoglou, Athanasios Papaioannou, Christos Sagonas, Jiankang Deng, Irene Kotsia, and Stefanos Zafeiriou. AgeDB: The first manually collected, in-the-wild age database. In *CVPR Workshops*, 2017.

Thao Nguyen, Haotian Liu, Yuheng Li, Mu Yi, Jiarui Mu, and Yong Jae Lee. Yo'LLaVA: Your personalized language and vision assistant. *arXiv preprint arXiv:2406.09400*, 2024.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. LibriSpeech: An ASR corpus based on public domain audio books. 2015.

Karol J Piczak. ESC: Dataset for environmental sound classification. In *ACM Multimedia*, 2015.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.

Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-Networks. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

Letta Team. Letta (formerly MemGPT): Stateful LLM agents with long-term memory. *arXiv preprint arXiv:2310.08560*, 2024. Software framework.

Yu Wang, Dmitry Krotov, Yuanzhe Hu, Yifan Gao, Wangchunshu Zhou, Julian McAuley, Dan Gutfreund, Rogerio Feris, and Zexue He. MemoryLLM: Towards self-updatable large language models. *arXiv preprint arXiv:2402.04624*, 2024.

WikiArt. WikiArt: Visual art encyclopedia. 2010. https://www.wikiart.org.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.

Yuhuai Wu, Markus N Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations (ICLR)*, 2022.